Súhrn OntoUML

pondelok 7. novembra 2022 23:11

Konceptuálne modelovanie - je činnosť popisujúca aspekty fyzického a sociálneho sveta za účelom porozumenia a komunikácie
Ontológia - filozofická disciplína, ktorá sa zaoberá existenciou, druhmi a štruktúrou objektov, vlastností, udalostí...

- V informatice se jako ontologie označuje reprezentace a definice kategorií, vlastností a vztahů mezi koncepty (pojmy, daty a entitami).
- Odkrývání a analýza obecných kategorií a principů, které popisují realitu, je předmětem formální ontologie.

Typy ontologií

-
  - **Vyšší ontologie (upper ontology)**, též **Základní ontologie (foundational ontology)**: univerzální, obecné abstraktní ontologie definující doménově a aplikačně nezávislé pojmy a vztahy (např. Unified Foundational Ontology -- UFO), které lze použít k přesnému zachycení a komunikaci o konkrétních doménách.
  - **Doménové ontologie (domain ontology)**: Ontologie popisující určitou konkrétní *doménu* (oblast) (např. těžbu ropy).
  - Doménové ontologie, které mají klíčový (obecný) význam pro určité odvětví jsou obvykle nazývány **klíčové ontologie (core ontology)** nebo **referenční ontologie (reference ontology)**

Modularizaci domény využívame abychom se vypořádali se složitostí a komplexností domény.
Pořadí činností při vytváření business informačního systému - analýza business dat, procesů, uživatelů - konceptuální model - implementační model - implementace
Rozdiel medzi inštanciou v UML a v OntoUML

- v UML platí, že inštancia je vždy jedna trieda, novú inštanciu vytvoríme cez new
- v OntoUML - inštancia sa zväčša skladá z viacerých tried, t.j. nejde o väzbu trieda-inštancia 1:1

OntoUML pracuje v **modálnej logike**. Modální logika nám umožňuje rozlišovat mezi neměnnými klasifikacemi a relacemi (rigid) versus klasifikacemi a relacemi, které se mohou měnit v závislosti na kontextu (anti-rigid).

- Modální logika pracuje s různými *světy*, resp. *situacemi* (konfigurace reality v čase a prostoru), s využitím kvantifikátorů:
  - ◇ = **možnost**: v některém světě (alespoň jednom) platí, že ...
  - □ = **nutnost**: ve všech světech platí, že ...

Unified foundational ontology (UFO)

- Byla a je používána k analýze, re-designu a integraci ontologických konceptuálních modelů v různých komplexních doménách.
- **UFO-A**: Strukturální aspekty (entity, vlastnosti, vztahy)
- **UFO-B**: Behaviorální aspekty (chování, události, kauzalita)
- **UFO-C**: Sociální aspekty (agenti, akce, organizace)

Kategorie typů objektů v OntoUML

- **Princip identity** říká, jakým způsobem identifikovat objekt v průběhu **celého života**, bez ohledu na jeho (libovolně drastické) změny.
- Základní rozdělení typů dle principu identity:
  - **Sortal Type** = poskytuje kategorizaci a má princip identity, tj. referenci a kvantifikaci.
  - **Non-Sortal Type** = nemá vlastní identitu z hlediska našeho vnímání.

Non-sortální typy

- Nemá vlastní identitu z hlediska našeho vnímání (v implementaci ji mít může: primární klíč, UUID, ...).
- Instance získá od non-sortálního typu **obecné vlastnosti** ale nikoliv princip identity.
- Všechny třídy reprezentující non-sortal typy jsou **abstraktní**.
- Instance non-sortal typu mohou mít **různé** principy identity.

Rigidný typ - ak je x inštanciou T v nejakom svete W, tak musí byť x inštanciou T v každom možnom svete W' - t.j. **nemôže** počas svojej existencie meniť svoj typ

- Rigidita je vlastností typu entity, která popisuje její proměnlivost.
- Non-rigidita je logickou negací rigidity.
- Rigidita vyjadřuje míru, s jakou jsou instance daného typu s tímto typem v rámci různých světů svázány.

Anti-rigidný typ - ak je x inštanciou T v nejakom svete W, tak je možné, že existuje nejaký svet W' kde x nie je inštanciou T - t.j. počas svojej existencie **môže** meniť svoj typ

Platí, že každý sortál musí mať práve jednu jednoznačnú identitu. Z čoho vyplýva, že:

- je zakázané dedenie, ktoré by nejakej entite umožňovalo mať viac identít
- ak entita nemá identitu musí ju od niečoho zdediť - anti-rigidné sortály musia mať v histórii dedenia práve jeden rigidný sortál

Typy, ktoré **poskytujú identitu** nemôžu mať ako predchodcu (priamo ani odvodene) typy, ktoré **poskytujú identitu** alebo typy, ktoré **nutne identitu dedia**. Porušil by sa princíp jednej identity, pretože by sa dedila ďalšia identita z predchodcu.

Non-sortal nemôže byť podtypom **sortálu** - v tom prípade by non-sortál zdedil identitu od sortálu. Non-sortál ale identitu nemá.
Rigidný typ alebo **semi-rigidný** typ nemôže byť podtypom **anti-rigidného** typu, pretože ak by anti-rigidný typ prestal existovať, prestal by existovať aj rigidný/semi-rigidný typ - čo nie je možné. Ale napríklad rigidný typ môže byť podtypom semi-rigidného typu.

Čo poskytuje identitu - ide všetko o rigidné sortály

- **Kind**
- **Collective**
  - <<memberOf>> Collective môžu byť len homogenné štruktúry - veci majúce rovnakú identitu - preto typy, ktoré agregujú rôznu identitu nemôžu byť podtypom Collective
- **Quality**
- **Relator**
- **Mode**
- **Quantity**

Typy, ktoré nutne dedia identitu - sortály, ktoré neposkytujú vlastnú identitu (musia mať identitu ale nevytvárajú si ju)

- **SubKind**
- **Role** - podlieha **relačnej závislosti** - musí mať povinný vzťah s nejakou inou entitou
- **Phase** - podmienka špecializácie je **intristická** (vnútorná) - určenie fáze sa robí podľa vnútornej vlastnosti inštancie

Typy, ktoré agregujú rôznu identitu - nemôžu mať ako predchodcu žiaden sortál, pretože by zdedili nejakú jednu identitu pričom oni agregujú rôzne identity v svojich podtypoch

- **RoleMixin**
- **PhaseMixin**
- **Mixin** - podtypom musí byť minimálne jeden rigid a jeden anti-rigid typ
- **Category**

Category - rigidný non-sortál

- používa sa k agregovaniu entín s **rôznou** identitou (zväčša Kind), nejde teda o homogénne podtypy (ako pri Collective)
- podtypom môže byť čokoľvek

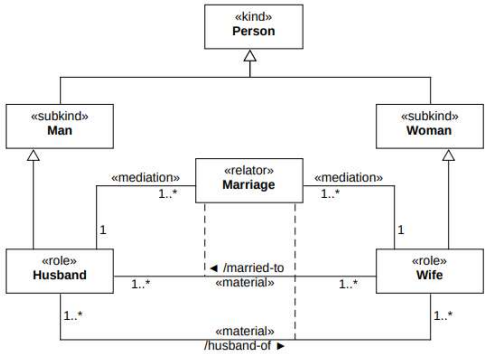
Relace

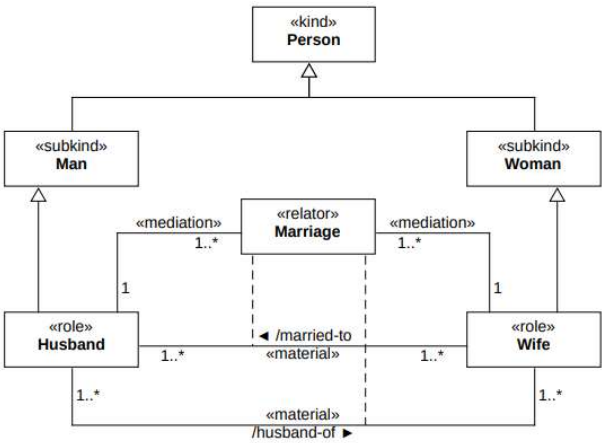
Formálne relace - spájajú dve alebo viaceré entity priamo bez ďalšieho zprostredkujúceho prvku. Vždy sme schopní identifikovať vnútorné vlastnosti, ktoré relaci definujú.

- ide vždy o odvodené relace (derived relation) - značené /názov_relace
- môžu byť použité k porovnávaniu entít

Materiálne relace

- existujú medzi dvoma entitami na základe nejakej zakladajúcej udalosti (founding event) - na základe tejto udalosti entita získala vlastnosti, ktoré sú existenčne závislé na druhej entite - to nazývame **qua individual**
- agregáciou všetkých qua individuals je entita - **relator** - agreguje teda všetky vlastnosti, ktoré vznikli po novom na základe tohto vzťahu
  - ide o pečatidlo (**truthmaker**) materiálnej relace - zmluva, svadba, spoločenská úmluva apod.
  - relator je **rigidný sortál poskytujúci identitu** - platia pre neho rovnaké pravidlá ako pre Kind
- <<**mediation**>> - je označenie relace medzi relatorom a entitami, ktoré spája (Role)
  - existenčne závislá relace
- <<**material**>> - označenie samotnej materiálnej relace
-





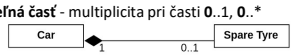
Vzťah celok-časť

- celok má vo väzbe pri sebe diamant (prázdny/plný), časť nemá
-



Povinnosť z hľadiska celku na časť

- **voliteľná časť** - multiplicita pri časti **0**..1, **0**..*
  -



- **povinná časť** - multiplicita pri časti **1**, **1**..*
  -



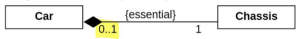
  - časť je povinná pre celok ale je možné ju vymeniť - t.j. pri výmene časti nedôjde ku zmene identity celku
  - napr. srdce človeka je možné meniť, nestratí sa jeho identita
  - označuje sa ako **generická závislosť** celku na časti
- **esenciálna časť** - multiplicita pri časti, 1, 1..*
  - **{essential}** - označí sa vzťah medzi celok-časť
  -



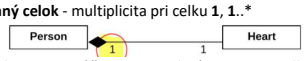
  - časť je povinná pre celok a pri výmene by sa zmenila identita celku - t.j. pre identitu celku je esenciálne, že táto **konkrétna** časť je časťou celku
  - napr. mozog človeka by pri výmene zmenil jeho identitu ako bytosti
  - označuje sa ako **existenčná závislosť** celku na časti
  - **len rigidné typy** môžu byť použité ako esenciálna časť
- povinnosť vs. esenciálnosť - človek musí mať **nejaké** srdce vs človek musí mať **konkrétni** mozog
- povinná časť je slabšia požiadavka, t.j. esencialita implikuje povinnosť

Povinnosť z hľadiska časti na celok

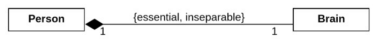
- **nepovinný celok** - multiplicita pri celku **0**..1, **0**..*
  -



  - (essential tam nemusí byť, pointa je to 0..1)
- **povinný celok** - multiplicita pri celku **1**, **1**..*
  -



  - ku existencií časti je potrebná existencia celku - t.j. celok je pre časť povinný avšak nemusí ísť o tento konkrétny celok
  - napr. teda srdce môže existovať len ak existuje aj človek ale môže to byť akýkoľvek človek - srdce sa vie transplantovať niekam inam
  - označuje sa ako **generická závislosť** časti na celku
- **neoddeliteľná časť** - multiplicita pri celku **1**, **1**..*
  - **{inseparable}** - označí sa vzťah medzi celok-časť
  -



  - trošku mind-bending ale stále sa na to treba pozerať z pohľadu povinnosti časti na celok - časť je neoddeliteľná vtedy keď pre svoju existenciu vyžaduje jeden **konkrétny** celok - celok je pre časť povinný a musí ísť o niečo konkrétne, nie akýkoľvek celok
  - napr. ak by neexistoval tento **konkrétny** človek tak nemôže existovať ani tento mozog - pretože keby vymeníme toho človeka tak už pôjde o iný mozog, resp. toho človeka nemôžeme vymeniť -> mozog je neoddeliteľnou časťou konkrétneho človeka
  - označuje sa ako **existenčná závislosť** časti na celku
  - **len rigidné typy** môžu byť použité ako neoddeliteľná časť - t.j. celok musí byť rigidný, časť môže byť aj anti-rigid

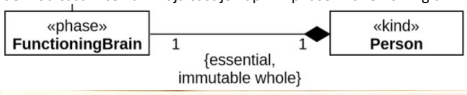
Immutable Part, Immutable Whole

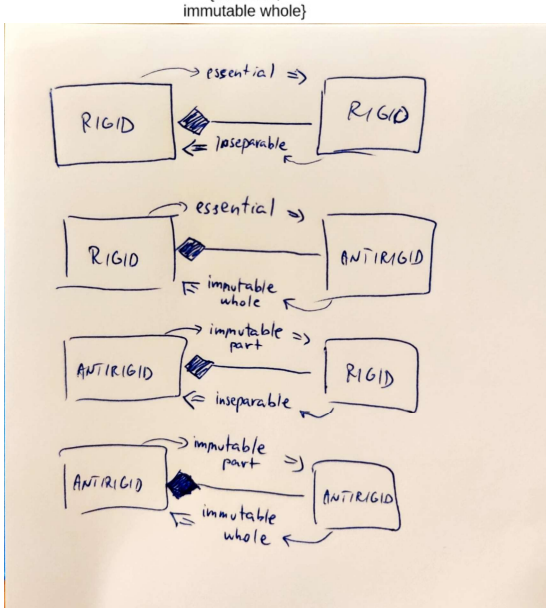
- je to isté čo essential a inseparable ale pre **anti-rigidné** typy:

	rigid	anti-rigid
celok -> časť	essential	immutablePart

časť -> celok	inseparable	immutableWhole

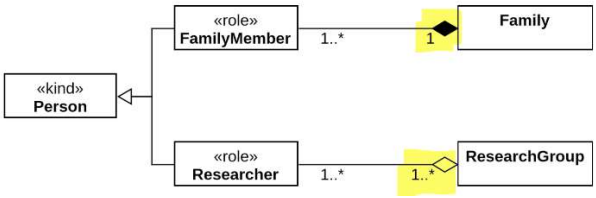
- idem od celku k časti - môj celok je <<kind>> čiže rigid -> musím používať rigid definície pre časť-celok, t.j. použijem {essential}
- idem od časti k celku - moja časť je napr. <<phase>> čiže non-rigid -> musím používať non-rigid definície pre časť-celok, t.j. použijem {immutableWhole}
-





Shareability

- ◆ = `{isShareable = false}` , nesdílitelná část, multiplicita max. 1
- ◇ = `{isShareable = true}` , sdílitelná část, multiplicita max. > 1



Typy agregací Celku a Částí

- Dosud jsme uvažovali `Kind` (či podtypy) jako agregující typ (celek).
- Studie kognitivních věd a lingvistiky však ukazují, že vnímáme vícero různých agregujících typů s různými vlastnostmi a vazeb Celek-Část:
  - Kvantita (`Quantity`)
    - `containment` : víno-láhev, malta-kyblík, jíl-socha, ...
    - `subQuantityOf` : alkohol-víno, písek-malta, křemík-jíl, ...
  - Kolektiv (`Collective`)
    - `memberOf` : strom-les, student-paralelka, pták-hejno, ...
    - `subCollectionOf` : severní cíp lesa-les, mladí ptáci-hejno, ...
  - Funkční celek (Functional Entity)
    - `componentOf` : srdce-oběhový systém, ředitel-firma, motor-automobil, ...

- Quantity a Collective sú **rigidné sortály**
- **Quantity** - skladá sa **z rovnakých častí ako celok** samotný - voda, alkohol...
  - je vymedzená **topologicky**, ide o **agregovaný celok**, ktorý je vždy **maximálny spojitý objekt**
  - je nekonečne deliteľný
  - <<**containment**>> - medzi Quantity a Kind - musí byť vzťah 1:1 medzi časťou a celkom
  - <<**subQuantityOf**>> - medzi dvoma Quantity - musí byť vzťah 1:1 medzi časťou a celkom a {essential}
- **Collective** - nie je zložený z častí rovnakého typu ako on sám ale **všetky prvky majú v kolektive rovnakú rolu**, sú ekvivalentné
  - vždy maximálne entita cez zjednocujúcu relaci, existuje **charakteristická/sjednocující relace** medzi entitami
  - <<**memberOf**>> - relace medzi Collective a prvkom - časť musí mať aspoň 2 prvky
    - irreflexívna, anti-symetrická, netranzitívna
  - <<**subCollectionOf**>> - relace medzi dvoma Collective - musí byť vzťah 1:1 medzi časťou a celkom a {inseparable}
    - irreflexívna, anti-symetrická, tranzitívna
  - kombinácia memberOf a subCollectionOf ale zachováva tranzitivitu
- **Funkční celek** - skladá sa z častí, ktoré majú **rôzne úlohy/funkcie v rámci celku**
  - <<**componentOf**>> - viaže časť k funkčnému celku
    - irreflexívna, anti-symetrická, tranzitívna v určitých prípadoch, inak nie

Objekty a aspekty

Objekty jsou nezávislé na všech ostatních objektech, které nejsou jejich esenciálními částmi nebo jichž nejsou oni neoddělitelnou částí.
Aspekty sú **existenčne závislé** na objekte, ktorému hovoríme **nositeľ (bearer)**. Táto väzba sa volá **inherence**.

- inherence je irreflexívna, anti-symetrická, non-tranzitívna
- pri zániku objektu (nositeľa) zaniknú aj všetky jeho aspekty, ktoré su na ňom závislé
- **intristický aspekt - Quality, Mode**
- **relačný aspekt - Relator**
- ide o **rigidné sortály** - poskytujú taktiež identitu iným objektom

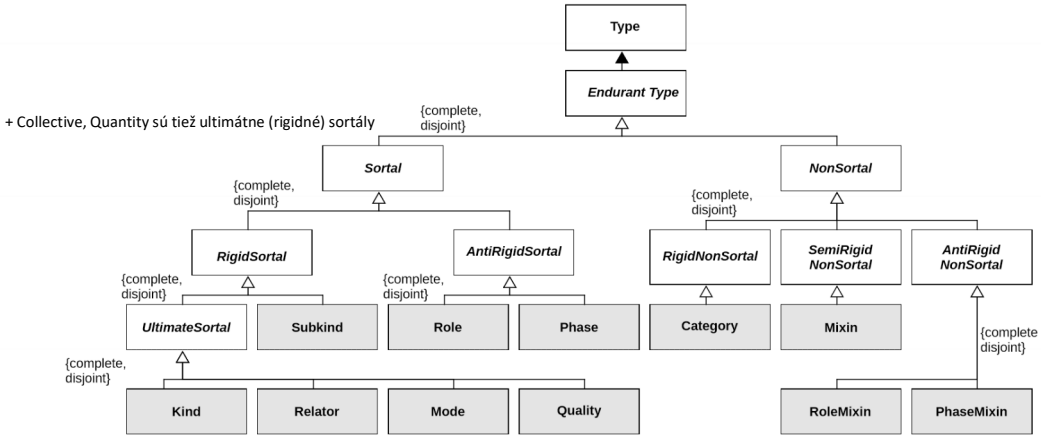
Quality - predstavujú merateľnú veličinu - farba, váha, rýchlosť apod.

- nemôže byť asociovaná s viacerými kvalitativnými doménami

Mode - nereprezentujú merateľné veličiny - schopnosť, vedomosť, vlasnosť apod.

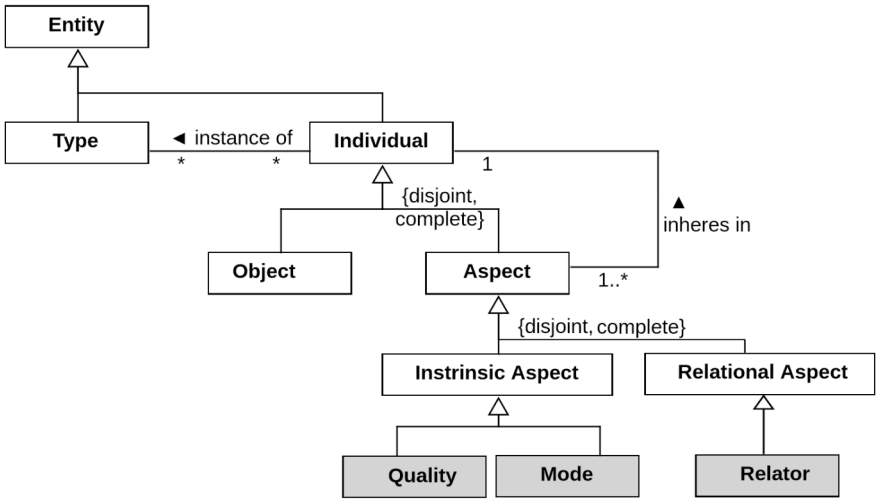
- Quality aj Mode sú intristicky závislé na objekte, t.j. musia mať pri nositeľovi multiplicitu **práve 1**, na strane Quality/Mode môže byť aj viac ako 1
- používa sa vzťah <<**characterization**>> medzi Quality/Mode a nositeľom

Shrnutí základních typů: OntoUML 2.0



Pozn.: Pozn.: *Endurant Type* jsou (zjednodušeně) strukturální typy. UFO-B se poté zabývá typy *Perdurant* konceptualizující děje (chování).

Shrnutí základních typů: Aspekty



Rigidní sortály

- «Kind» = základní rigidní sortál poskytující identitu (*identity provider*)
  - «Relator» = rigidní sortál poskytující identitu, *truthmaker* «material» vztahu
  - «Collective» = rigidní sortál poskytující identitu, celek se členy stejného druhu
  - «Quantity» = rigidní sortál poskytující identitu, celek bez rozlišení jednotlivých členů (např. tekutiny, sypké materiály, maziva a další)
- «SubKind« = rigidní sortál dědící identitu

Anti-rigidní sortály

- Musí (stejně jako «SubKind«) zdědit identitu (tranzitivně) z právě jednoho *identity provider* = každá instance musí mít **právě jednu identitu**
- «Phase» = pouze vnitřní vlastnosti, které se mohou v čase měnit
  - Vždy tvoří *phase partition*, který musí být `{disjoint, complete}`
  - Například: `Alive` a `Deceased`
- «Role» = vlastnosti závisející na další entitě, které se mohou v čase měnit
  - Vždy musí mít **povinný vztah**, který "potvrzuje", že je instance *v roli*
  - Například: `Student`, `Employee` nebo `Committee`

Non-sortály

- Nadtřídy pro společné vlastnosti entit **různých druhů** = různých principů identit
- Vždy se jedná o **abstraktní třídy** = název kurzívou
- «Category» = rigidní non-sortál, pro rigidní podtypy
- «PhaseMixin» = anti-rigidní non-sortál, pro «Phase» různých druhů
- «RoleMixin» = anti-rigidní non-sortál, relačně závislý, pro «Role» různých druhů
- «Mixin» = semi-rigidní non-sortál, pro rigidní **a současně** anti-rigidní podtypy

- subQuantity musí mať aj {essential}

Aspekty

- *Slabé* entity, vždy závislé a náležící svému nositeli (*bearer*), typicky významné atributy
- «Quality» = měřitelné veličiny
  - Například: `Height`, `Color`, `Density`, `Volume` (objem i hlasitost)
- «Mode» = neměřitelné veličiny a stavy
  - Například: `Symptoms`, `Skill`, `Expertise`, `Book Edition`, ...

Asociace

- Stejně jako třídy, tak i vztahy musí mít v OntoUML blíže určený typ
- «formal» = vztah odvozený přímo z vlastností entit ve vztahu
- «material» = vztah odvozený z další entity, která je *truthmaker* vztahu
  - *truthmaker* = třída se stereotypem «Relator» (či podtřída)
  - «Relator» pattern využívající «Role» na obou stranách vztahu
  - multiplicity pro «material» a odpovídající «mediation» musí být konzistentní
- «containment» = vztah mezi «Quantity» a kontejnerem, vždy 1-1
- «characterization» = vztah mezi aspektem a nositelem, vždy 1 na straně nositele
- «derivation» = vztah mezi «material» a «Relator» (čárkovaná čára)

Vztahy celek-část

- Diamant na straně celku, plný = nesdílitelné části, prázdný = sdílitelné mezi více celky
- «memberOf» (**M**) = mezi členy a «Collective», vždy min. 2 na straně části
- «subCollectionOf» (**C**) = mezi «Collective», vždy 1-1 kvůli *maximalitě*
- «subQuantityOf» (**Q**) = mezi «Quantity», vždy 1-1 kvůli *maximalitě*
- «componentOf» = mezi částí a funkčním celkem

Omezení Generalization Set

- (bez omezení) = instance lze tvořit z nadtřídy a libovolné kombinace podtříd
- `{disjoint}` = instance nadtřídy nebo (xor) jedné z podtříd
- `{complete}` = instance libovolné kombinace podtříd (ne pouze nadtřídy)
- `{disjoint, complete}` = instance právě jedné z podtříd

Omezení vztahů celek-část

- `{essential}` = celek nemůže existovat bez stále stejné instance části
- `{inseparable}` = část nemůže existovat aniž by byla částí stále stejného celku
- V případě anti-rigidity chceme zajistit **neměnnost** při ztrátě a znovu-získání vlastností (např. změna fáze *tam a zpět*)
- `{immutablePart}` = podobné `{essential}`, ale pro anti-rigidní celek
- `{immutableWhole}` = podobné `{inseparable}`, ale pro anti-rigidní část